CS781 Capstone Project: In-House Legal Intelligence System

Aryan Kumar
261110012
aryank26@iitk.ac.in
Indian Institute of Technology Kanpur (IIT Kanpur)

Abstract

Law firms cannot send confidential judgments to hosted models, so we built a self-hosted Indian legal NLP stack under a 300M-parameter cap, trained only on the course data. We first trained four 63–67M-parameter decoders on about 125M legal tokens, then restarted and trained three 214M-parameter models on 4.0B tokens each: a grouped-query attention (GQA) decoder, a Hymba-style hybrid of attention and Mamba-2 heads, and a masked-language-model encoder. Every later phase (entity recognition, rhetorical roles, judgment prediction, statute identification, summarization and prior case retrieval) fine-tunes one of these checkpoints with full fine-tuning, LoRA, LoRMA, IA^3, partial tuning or distillation. Three findings recur. More pretraining helped: the 214M models beat the 67M ones by about 0.05 macro-F1 on statutes. Architectures and adaptation methods tie within seed or test noise, and LoRMA matches full fine-tuning at about half the memory. Lexical baselines are hard to beat, and word n-gram overlap carries most of our retrieval score.

1 Competition Result

Codabench username: a_261110012 **Roll number:** 261110012

Table 1 lists our best valid score in each phase.

Table 1: Official phase scores and ranks (rank of our leaderboard entry among all participants), recorded on 8 November 2026 at 21:10 IST. Phase 7 was still open until 23:59 IST.

Phase	Task	Primary metric (value)	Rank	Valid submissions
1	Legal LM Pretraining	BERTScore-F1 0.8298	84 of 142	11 (7 distinct)
2	Legal NER	strict macro-F1 0.5443	52 of 132	7 (6 distinct)
3	Rhetorical role prediction	macro-F1 0.4642	89 of 131	8
4	Court judgment prediction	macro-F1 0.5992	122 of 130	18
5	Legal statute identification	macro-F1 0.3965*	41 of 129	14
6	Abstractive summarization	BERTScore-F1 0.8450	42 of 124	12
7	Prior Case Retrieval (PCR)	Micro-F1@7 0.4938	8 of 121	17

*A later Phase 5 submission scored 0.4180 but is marked "Failed" and is not on the leaderboard.

Total valid Codabench submissions across all phases: 87 **Date ranks were recorded:** 8 November 2026

We met the minimum of four distinct submissions in every phase.

2 Introduction and Problem Description

A law firm's documents are confidential, so it needs language models that run on its own hardware and were trained only on data it controls. The capstone builds such a stack for Indian law in seven phases: pretraining a 50M–300M-parameter foundation model from scratch and continuing 128-word judgment prompts (Phase 1, BERTScore F1 [1]); entity spans of 12 types with exact character offsets (Phase 2, strict macro-F1); one of 13 rhetorical roles per sentence (Phase 3, macro-F1) [2]; whether an appeal is accepted (Phase 4, macro-F1) [3]; the set of 100 statutes a case invokes (Phase 5, macro-F1); a summary of a judgment (Phase 6, BERTScore F1); and the prior cases a judgment cites (Phase 7, Micro-F1 over seven predictions).

No pretrained model or external corpus may be used, so every neural component descends from our own Phase 1 checkpoints. Compute was Colab compute units, which fixed model size and token budget, and judgments (median 4,200 tokens in Phase 4) are far longer than the 512 or 2,048 tokens our models read. We asked four questions throughout: does more pretraining help, do attention variants or a state-space hybrid help, which adaptation method transfers a checkpoint best, and how far does a neural model beat a lexical baseline. We built and submitted every phase end to end. We kept the course tokenizer rather than training our own, and did not exploit the Phase 7 citation traces (Section 3.3).

3 Data Analysis and Preprocessing

3.1 Sources and official splits

We used only the official configurations of `Exploration-Lab/CS-781-Capstone` (Table 2).

Table 2: Splits and design-relevant statistics (our counts). Lengths are in course-BPE tokens unless marked as words.

Phase	Train / dev / test	Statistics	Design consequence
1	19.2 GB text	High Courts 10.67 GB, tribunals 6.48, Supreme Court 1.32, Acts 0.73; 3.7–4.0 characters per token; 4.3–4.6B tokens after cleaning	One pass in natural proportions; 4.0B-token budget, about 19 tokens per parameter
2	70 docs (2 folds) / – / 35 docs	Median 4,488 words; 120 entities per document. DATE 18.8%, STAT 15.0%, PREC 14.1%, …, counsel and witnesses about 2% each. Median span 2 words (p95 9); no overlapping spans; 26 strings carry two labels ("Union of India")	BIO tags over words; CRF with legal transitions; windows over the full text
3	80 / 10 / 10 docs (16,845 / 2,142 sentences)	13 roles, CL and IT domains. ArgumentPetitioner is 24.6% of dev CL sentences against 9.2% in train; 17.6% (CL) and 22.2% (IT) of sentences exceed 63 tokens	Neighbouring-sentence context; square-root class weights
4	31,677 / 994 / 1,517	Train is 41% accepted, dev 50%. Mean 5,500 tokens (median 4,200, p90 10,000). TF-IDF on the last third scores 0.629, the first third 0.576	Read the last eight 512-token chunks
5	42,513 / 10,155 / 13,019	100 statutes with 272–11,731 training cases each; 3.7 per case (max 31). Mean 1,909 tokens, median 768	Eight chunks cover 87% of cases; per-statute thresholds
6	7,030 / 200 held out / 100	Judgments: median 3,129 words (p90 7,855). Summaries: median 638 words, ratio 0.217; 11% of summary sentences occur verbatim	Abstractive decoder; length set as a ratio of the input
7	827 / 118 / 237 queries; pools 4,320 / 1,023 / 1,727	Gold per dev query: mean 8.2, median 5, max 92; a perfect ranking scores 0.618. Gold is older than the query in 352 of 352 dated pairs	Lexical first stage; date filter; self-matches removed

3.2 Corpus statistics that actually informed design

Phase 1. High Court and tribunal text is 89% of the corpus. The first models sampled sources equally but saw only 2.7% of the corpus, so the restart made one pass in natural proportions and spent the budget on tokens rather than balance. **Phase 3.** Dev has only 10 documents and its CL part lacks Dissent and PrecedentNotReliedUpon, so dev macro-F1 averages present classes and moves by points with one document. **Phase 4.** Every dev case appears twice (1,988 rows), and 5,710 duplicate texts were removed from train. The last third of a case is the most predictive, so we keep the last chunks. **Phase 5.** The 43-fold gap in statute frequency makes one global threshold too high for rare statutes. **Phase 7.** Queries with one or two gold cases cap precision near 0.2, and those with more than seven cap recall.

3.3 Preprocessing

All phases use the frozen course BPE tokenizer (vocabulary 32,768) without lowercasing or other normalisation. For Phase 1 we dropped empty documents, documents under 200 characters and exact duplicates. We also ran a contamination audit: we sampled token 16-gram fingerprints from the Phase 4 and 5 dev and test texts and dropped the 5,470 corpus documents that shared 20 or more of them. We never ran this audit for Phases 2 and 3. **Phase 2 offsets:** the tokenizer returns, for every BPE piece, its character span in the original text (Unicode code points, end exclusive). We never normalise the text, so predicted words map back to exact characters, and we checked that every gold offset slices out its gold string (0 mismatches in 8,393 entities). **Phase 3** uses the provided sentence lists unchanged and caps each sentence at 63 tokens. **Phases 4 and 5** split a case into 512-token chunks and keep at most eight (the last eight in Phase 4, eight spread uniformly in Phase 5). **Phase 6** fills a document budget (1,536 tokens in the final system) with the first three quarters from the start of the judgment and the last quarter from its end, and gives the rest of the context to the summary. **Phase 7** gives query and candidate one 512-token excerpt each: the 96-token opening plus the sentences sharing the most rare word 4-grams with the other document. The organisers mask citations as `<CITATION>`, but traces survive the masking. Matching them directly raised dev F1 from 0.4603 to 0.4747, but that reads answers from gaps in the masking and improves no model, so we never submitted it.

4 Approach and Model Evolution

4.1 Phase-1 foundation model

First models. Our first four models were 12-layer causal decoders (width 512, 8 heads, SwiGLU FFN of 2,048 [4], RMSNorm [5], RoPE [6], context 2,048) differing only in attention and normalisation: multi-head with pre-norm (67.4M), GQA with 2 key–value heads [7] (62.7M), latent attention [8] (67.0M) and multi-head with post-norm (67.4M). After a WikiText stage they saw about 125M legal tokens with sources sampled equally, and they served Phases 2 to 5. Our Task 1 submissions came from them (Table 7); greedy decoding with repeated token 4-grams blocked scored best (0.8298 BERTScore). The post-norm model repeated itself in Task 1 and collapsed under partial fine-tuning later, and no model beat TF-IDF in Phases 4 and 5. A frozen-feature probe on Phase 4 data gave 0.562 dev macro-F1 for the old GQA model against 0.514 for random weights and 0.612 for TF-IDF. We read this as under-training and restarted on 27 October.

Final models. We trained three models of the same size on the same 4.0B tokens in the same order, so their losses are directly comparable (Table 3, Figure 1). The GQA decoder has 16 layers, width 1,024, 16 query and 4 key–value heads and a SwiGLU FFN of 2,816. The hybrid follows

Hymba [9]: in every layer, eight GQA heads and eight Mamba-2 SSD heads [10] read the same normalised input in parallel, each output is normalised and scaled per channel by a learned vector, and the two are concatenated before the output projection. We omitted Hymba's meta tokens, sliding-window attention and cross-layer key–value sharing, so we call it Hymba-style; our plain-PyTorch SSD scan is tested against the step-by-step recurrence. The encoder reuses the decoder code with bidirectional attention and a masked-language-model loss at 30% masking [11], on 512-token windows. We verified the parameter bound by summing parameter sizes at construction (213.9M, 215M and 213.9M). We dropped a latent-attention decoder after a benchmark: it ran at 56k tokens/s against 130k for GQA, so it would have cost about 177 compute units.

All three use AdamW [12] ($\beta = (0.9, 0.95)$, weight decay 0.1 on matrices only), a peak learning rate of 6×10^{-4} with 300 warm-up steps and cosine decay to 6×10^{-5}, gradient clipping at 1.0, bf16 and batches of 524,288 tokens for 7,629 steps. The data order is a fixed function of the step, so a killed run resumes on exactly the unseen data. The budget of about 19 tokens per parameter follows the compute-optimal ratio [13].

Table 3: Final Phase 1 models, all trained on the same 4.0B tokens. The encoder's masked-LM loss is not comparable with the causal losses.

Model	Shape	Params	Speed (tokens/s)	Wall clock	Final val. loss
GQA decoder	16 layers, d 1024, 16/4 heads, FFN 2816	213.9M	130k	8.6 h	1.236
Hybrid	16 layers, 8 GQA + 8 SSD heads, FFN 2688	215M	113k	10.0 h	1.189
MLM encoder	as GQA, bidirectional, window 512	213.9M	172k	6.5 h	0.686 (MLM)

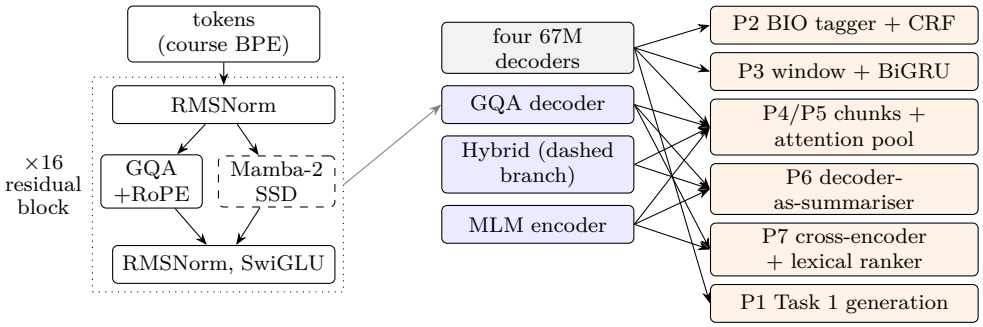


Figure 1: Left: the shared block; the hybrid adds the dashed SSD branch beside attention. Right: which checkpoint each phase fine-tunes. Phase 4 and 5 arrows from the new models refer to the probe (Phase 4) and the final runs (Phase 5).

4.2 Transfer protocol (Phases 2–7)

Every neural task model starts from a Phase 1 checkpoint with a small task head; only the TF-IDF baselines and the Phase 7 ranker are not language models. We compared six ways to adapt a checkpoint. Full fine-tuning updates all weights. Partial tuning updates only the top blocks. LoRA [14] adds a trainable low-rank update $W + BA$ to the query and value projections (rank 8 in Phase 3, 16 later). LoRMA [15] applies the low-rank term multiplicatively, $(I + BA)W$, so it rescales existing directions rather than adding new ones. IA^3 [16] learns only per-channel scaling vectors. Distillation [17] trains a student that keeps every other block of the same checkpoint (6 of 12 or 8 of 16) on a mix of the labels and the full fine-tuned teacher's softened outputs (temperature 2, weight 0.5). Section 5.3 compares these methods across phases.

4.3 Phase 2 - Legal NER

We treat NER as BIO tagging over words (25 tags). Each judgment is cut into 512-token windows with stride 384, a word is represented by the backbone state of its last BPE piece, and a BiLSTM over these states produces tag scores. A CRF [18] whose transition matrix forbids illegal moves (an I tag after O or after a different type) decodes each window with Viterbi, so every predicted span is well formed and many spans per document come out at once. A span runs from its first word's first character to its last word's last character. We fine-tuned the GQA and MHA pre-norm models, trained GQA longer (15 epochs), tried LoRMA and a 6-layer distilled student, and finally averaged the two models' tag scores and CRF parameters with equal weight. A span classifier over candidate spans of up to 32 tokens was built but never submitted.

4.4 Phase 3 - Rhetorical role prediction

A window of 12 consecutive sentences, each capped at 63 tokens, is packed with three context sentences on each side into one sequence of at most 512 tokens. Each sentence is represented by its mean and last token states, which pass through an MLP, a bidirectional GRU over the window and a linear role classifier, trained with square-root inverse-frequency class weights. One model covers both domains. A sentence-only ablation encodes each sentence alone. We tried full fine-tuning (MHA), LoRMA (GQA), LoRA (MLA) and IA^3 (MHA), position features and a distilled LoRMA student taught by the MHA model. Document-level transition priors and CRF layers on cheap features did not help (Section 7).

4.5 Phase 4 - Court judgment prediction

The last eight 512-token chunks of a case are encoded separately and pooled with learned attention weights into one case vector for a binary head. Every run is submitted at threshold 0.5 and at a dev-tuned threshold. We trained nine runs that cross architecture with method, a 6-layer distilled student, and an equal-weight ensemble of runs 1 to 4 chosen before seeing test.

4.6 Phase 5 - Legal statute identification

The Phase 4 encoder, with eight uniformly spread chunks, feeds 100 independent sigmoid outputs trained with binary cross-entropy and square-root positive weights. Per-statute thresholds t_k overfit for rare statutes, so we shrink them towards the global threshold g:

$$\theta_k = \lambda_k t_k + (1 - \lambda_k)\, g, \qquad \lambda_k = \frac{n_k}{n_k + m}, \tag{1}$$

where n_k is the number of training cases with statute k; $m = 0$ gives per-statute thresholds and $m \to \infty$ one global threshold. We chose m with five random two-fold cross-fits on dev and always predict at least one statute. The final system averages six neural runs (three new models, two seeds) with a TF-IDF linear SVM, weighting TF-IDF by 0.9.

4.7 Phase 6 - Abstractive summarization

Each model is a decoder-only summariser over "document, end token, 'Summary:', summary", with loss only on summary tokens. The encoder runs as a prefix language model, bidirectional over the document and causal over the summary. Summary length is 0.18 of the document's word count, clipped to 200–700 words, decoded by 4-beam search with no repeated 3-gram. In the final system

the document gets 1,536 of the 2,048 context tokens, leaving about 510 for the summary, which is about the span BERTScore reads (Section 7). We did no post-training or alignment.

4.8 Phase 7 - Prior Case Retrieval (PCR)

The system has three stages [19]. A hashed word 4-gram overlap score (document-frequency cap 0.05, length normalisation 0.6) orders the pool after removing self-matches and judgments dated after the query. It scores 0.4603 dev F1 against 0.101 for unigram BM25 [20], because a citing judgment often quotes the cited one. A gradient-boosted ranker [21, 22] rescores the top 50 from 27 features. They include overlap computed only from query sentences near a `<CITATION>` placeholder, which uses where a citation sits, not which case it was. Finally, Phase 1 rerankers read the paired excerpts (Section 3.3) of each top-50 candidate jointly, and a linear head on the token mean (encoder) or last token (GQA) scores the pair, trained with softmax cross-entropy over one gold case and seven negatives from the lexical top 40. The final score is

$$s(q,c) = z_q\big(\operatorname{logit} r(q,c)\big) + 0.2 \cdot \tfrac{1}{3}\textstyle\sum_{m=1}^{3} z_q\big(n_m(q,c)\big), \tag{2}$$

where z_q standardises within a query, r is the ranker and n_m are three rerankers: a fully fine-tuned encoder on the plain excerpts, and a fully fine-tuned encoder and a GQA LoRMA model on query excerpts filled first from citing sentences. Four ideas failed: the neural score as a ranker feature, retraining on train and dev, four-epoch training (train top-1 accuracy 0.95, dev neural F1 near 0.32), and averaging eight rerankers (test 0.4932).

5 Experiments and Implementation Details

5.1 Compute and training protocol

All training ran on Colab GPUs (T4, L4, A100 and G4). Pretraining cost about 77 compute units for GQA and 87–90 for the hybrid; a Phase 7 reranker run costs about 8. Task runs use AdamW with warm-up and decay, bf16, separate learning rates for backbone, adapters and head, and a wall-clock cap with resumable checkpoints. Phases 3 to 5 keep the epoch with the best dev macro-F1, Phase 6 the best validation loss, and Phase 7 the best dev score of the reranker alone. Table 4 gives the settings of each phase's final model.

Table 4: Hyperparameters of the final model in each phase. LR: backbone / adapter / head.

	P1	P2	P3	P4	P5	P6	P7
Model	GQA	GQA + MHA	GQA LoRMA	4-run ens.	3 models	hybrid	2 enc. + GQA
Parameters (M)	213.9	62.7 + 67.4	62.7	39.9–67.4	214–215	215	213.9–214.6
Context length	2048	512, stride 384	≤512	8×512	8×512	2048	512+512
Peak LR	6e-4	1e-5/–/1e-3	–/1e-3/1e-3	3e-5/1e-3/5e-4	5e-5/–/1e-3	3e-5	2e-5 (3e-4)/1e-4
Batch × accum.	32×8 seq.	1 window	8 windows	4×2 cases	4×4 cases	2×8	8 pairs×4
Epochs / steps	7,629 steps	best of ≤20: 6, 1	8	5×2,000 cases	4×12,000 cases	2	2 or 4
Dropout	0	0.2	0.1	0.1	0.1	0	0
Decoding / threshold	greedy, no-rep. 3	BIO, Viterbi	argmax	dev-tuned	shrunk, m=30	4 beams, no-rep. 3	top 7

5.2 Local evaluation

Phases 3, 4, 5 and 7 have labelled dev splits (Phase 4 deduplicated to 994 cases); for Phase 6 we held out 200 training documents and scored 100 with the organisers' BERTScore settings. Phase 5 per-statute thresholds are scored by two-fold cross-fit, and Phase 7 systems are compared with a paired bootstrap over queries. Local and Codabench scores diverged twice. Phase 4 dev-tuned thresholds overstated test by 2.5 points on average and beat threshold 0.5 by only 0.12 points on

test. Phase 6 test ran 0.005–0.008 below validation for every submission, probably because test references are longer (932 against 751 words), but gains carried over. Phase 7 test tracked dev within about 0.006, but Phase 3 dev (10 documents) did not rank runs reliably (Section 7).

5.3 Ablations and controlled comparisons

Adaptation method. Table 5 collects every run that changed the adaptation method. Within each phase the differences are inside the measured noise: test standard error about ±0.013 in Phase 4, seed spread 0.003–0.007 in Phase 5 and 0.0097 in Phase 7. The Phase 4 controls that hold architecture fixed (runs 2, 7, 8) or method fixed (runs 2, 9, 10) show no effect on test. LoRMA matched full fine-tuning in Phases 4, 6 and 7 with under 1% of the weights trainable and about half the memory (6.2 against 12.0 GB in Phase 7). Partial tuning of the post-norm model collapsed in every phase. The distilled student matched its teacher in Phase 4 (0.5766 against 0.5776 dev) with 41% fewer parameters and 2.2 times faster inference, but lost a third of standalone quality in Phase 7.

Table 5: Adaptation methods across phases. Backbones: G = GQA, M = MHA pre-norm, P = MHA post-norm, L = latent attention (old 67M models in P3–P5), E = encoder (new 214M models in P6–P7, with G). P3: dev macro-F1. P4: test macro-F1 at threshold 0.5. P5: old models, dev macro-F1, global threshold. P6: test BERTScore. P7: dev neural F1 when the model alone ranks the top 50 (chance 0.112).

Method	P3	P4	P5	P6	P7
Full fine-tuning	0.4622 M	0.5805 M, 0.5838 G	0.305 M	0.8350 G, 0.8345 E	0.3168 G, 0.3391 E
LoRA	0.4457 L	0.5665 L, 0.5648 G	0.279 L	0.8303 G	0.3135 G
LoRMA	0.4478 G	0.5854 G, 0.5854 M	0.268 G	0.8330 G	0.3335 G
IA^3	0.4153 M	–	–	–	0.3002 G
Partial (top blocks)	–	0.4877† P	0.088 P	–	0.3168 G
Distilled student	0.4942‡ G	0.5846 G (6 of 12)	0.261 G	0.8299 G (8 of 16)	0.2079 G (8 of 16)

†Dev-tuned threshold. ‡LoRMA student with all 12 blocks, taught by the MHA model: distillation as regularisation, not compression.

Pretraining scale and architecture. Moving from the 67M models to the 214M models raised Phase 5 dev macro-F1 from 0.305 to 0.354–0.363, the largest single gain in the project. The frozen probe moved from 0.562 (old GQA) to 0.589 (new GQA) and 0.602 (hybrid at 75% of training), still below TF-IDF (0.612). The three new models are indistinguishable downstream: Phase 5 dev cross-fit 0.3703, 0.3667 and 0.3637 for encoder, hybrid and GQA, within a seed spread of up to 0.007, and Phase 6 test 0.8356 (hybrid) against 0.8350 (GQA).

Phase 7 controls. Table 6 changes one training factor of the encoder reranker at a time. We call a difference real only if its interval excludes zero and it clearly exceeds the seed spread.

Table 6: Phase 7 one-factor controls on dev, one seed each. Neural F1 as in Table 5; intervals from a paired query bootstrap. Hits: gold cases in the top 7 when the run is added to the ranker at weight 0.2 (ranker alone 445). *Mean pooling, hard negatives, listwise loss.

Run	Neural F1	Difference [95% interval]	AUC (top 50)	Hits
Encoder baseline*	0.3391	–	0.771	447
Pairwise loss	0.3324	−0.0067 [−0.0173, 0.0035]	0.765	447
Random negatives	0.4002	+0.0611 [0.0363, 0.0886]	0.881	442
Last-token pooling	0.3079	−0.0311 [−0.0501, −0.0135]	0.747	444
Frozen backbone, head only	0.2379	−0.1012 [−0.1282, −0.0757]	0.676	447
Random start, no Phase 1	0.1946	−0.1445 [−0.1784, −0.1119]	0.629	444
GQA baseline (last token)	0.3168	–	0.756	442
GQA mean pooling	0.3357	+0.0189 [−0.0021, 0.0406]	0.779	444

Phase 1 transfer is real for the standalone model (random start 0.1946, frozen 0.2379, fine-tuned 0.3391), and the encoder's lead over GQA is mostly pooling (0.3391 against 0.3357 with mean pooling on both). But the frozen backbone ties the fine-tuned one at 447 blended hits, so the blended gain cannot be credited to fine-tuning. Random negatives gave the best standalone model and the worst blend (interval $[-0.0110, -0.0011]$ against the baseline blend, uncorrected); their scores correlate 0.555 with the ranker against 0.245 for hard negatives, so they seem to learn topical relatedness the ranker already has.

6 Results

Table 7: Main variants per phase, in the order we built them. Test: Codabench primary metric. Dev: our split (Phase 6: validation BERTScore).

Phase	Variant	Dev	Test
1	First models, greedy: post-norm / latent / MHA / GQA	–	0.737 / 0.795 / 0.798 / 0.799
1	Nucleus sampling (p 0.9, T 0.8); greedy + no repeated token 4-gram	–	0.8211; 0.8298
2	BIO + BiLSTM-CRF: MHA (epoch 1) / GQA (epoch 6) / GQA (epoch 15)	–	0.4647 / 0.5388 / 0.5417
2	Distilled 6-layer student; GQA LoRMA	–	0.5054; 0.0155
2	GQA + MHA ensemble, equal weights (best)	–	0.5443
3	Sentence alone, GQA LoRMA	0.3414	0.3532
3	Sentence window + BiGRU, MLA LoRA / MHA full	0.4457 / 0.4622	0.4509 / 0.4508
3	Window, GQA LoRMA and MHA IA^3 (best submission)	0.4478 / 0.4153	0.4543, 0.4642
3	Later runs: distilled student, position features	0.4511–0.4942	0.4294–0.4386
4	Best single run at threshold 0.5 (GQA LoRMA; MHA LoRMA)	0.5966; 0.5866	0.5854
4	Equal ensemble of runs 1–4, tuned threshold	0.6245	0.5992
4	Ensemble of runs 1–2 picked on dev, tuned threshold	0.6388	0.5912
5	Old MHA full fine-tune, per-statute thresholds	0.314	0.3129
5	Leaderboard entry: earlier tuned ensemble (TF-IDF weight 0.5, $m = 30$)†	0.3978	0.3965
5	Six new runs + TF-IDF (weight 0.9), shrinkage $m = 30$	0.4122	0.4180*
6	Lead baseline (first k words)	–	0.8256
6	Hybrid, greedy, ratio 0.22	–	0.8356
6	+ ratio 0.18, 4 beams	0.8469	0.8409
6	+ document budget 1,536 tokens (final)	0.8492	0.8450
7	Word 4-gram overlap	0.4603	0.4554
7	+ learned ranker (19 features)	0.4847	0.4790
7	+ citing-context features (27 features)	0.4947	0.4891
7	+ three rerankers (final)	0.4992	0.4938

*Scored but marked "Failed" by Codabench; not on the leaderboard. †Most likely configuration; the zip was overwritten and could not be matched exactly.

Phase 1. The hybrid's validation loss was lower than GQA's at every evaluation (Figure 2); the gap peaked at 0.106 and narrowed to 0.047 at 4.0B tokens. We tested the new GQA model's decoding offline on 60 official prompts with our own ROUGE. Greedy decoding looped in 86% of outputs; blocking repeated 3-grams raised ROUGE-L from 0.216 to 0.226, ahead of nucleus sampling (0.213) and a repetition penalty (0.203), yet only just beat copying the prompt's last 128 words (0.198). We never submitted these continuations.

Phases 2–6. In Phase 2 the ensemble added only 0.006 over GQA alone. Context mattered most in Phase 3: the sentence window added about 10 points of test macro-F1. In Phase 4 the ensemble fixed before seeing test (0.5992) beat the pair chosen on dev (0.5912), and in Phase 5

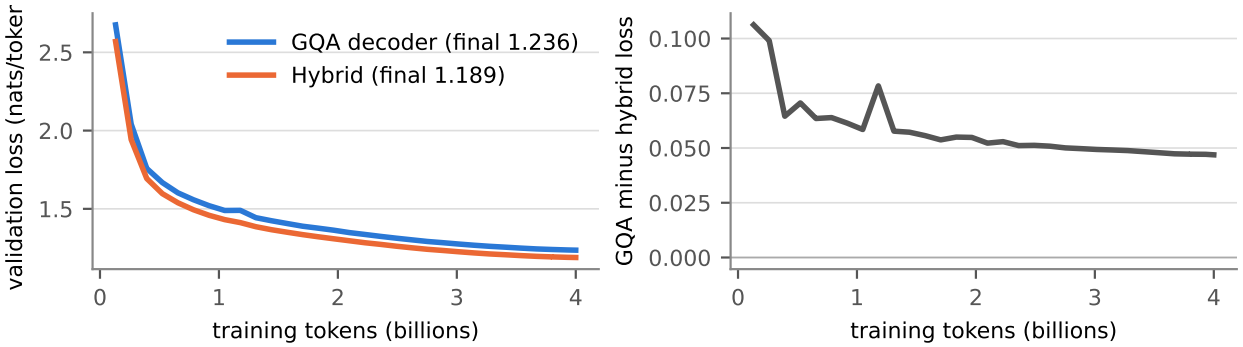


Figure 2: Phase 1 validation loss for the GQA decoder and the hybrid on the same data order (left), and the gap between them (right). The hybrid needed about 20% fewer tokens to reach GQA's loss at step 1,250, and finished 0.047 lower.

the gain came from the new pretraining and the TF-IDF ensemble, not the architecture. In Phase 6 shorter outputs scored higher (ratio 0.18 beat 0.22), beam search added 0.0053 on test with six beams no better than four, and the final budget split gave the best BERTScore while test ROUGE-1 fell from 0.484 to 0.459, so the two metrics disagree.

Phase 7. Our best entry scored 0.4938 Micro-F1@7 (precision 0.5039, recall 0.4841). Only the learned ranker was clearly significant (+0.0245 dev, interval [0.0095, 0.0410]); the citing-context features added +0.0100 ([−0.0010, 0.0214]) and the rerankers +0.0044 ([0.0010, 0.0094]), about eight test hits, with the blend weight chosen from about 15 dev combinations.

What won, and why. In the classification phases the winner was the system that read the most relevant text with the best-trained backbone, combined with a lexical model. The 214M models helped because they saw 30 times more legal text, and the hybrid's lower loss did not turn into a measurable downstream gain. In retrieval the winner was exact text reuse: citing judgments quote the cases they cite, and a word 4-gram overlap captures that directly, while a 1,024-token neural excerpt only partly preserves it.

7 Error Analysis and Discussion

7.1 Phase-wise failure modes

Phase 2. The ensemble's gain over GQA alone was small (0.5443 against 0.5388), and training GQA from 6 to 15 epochs added only 0.003. LoRMA scored 0.0155, far below every other run, which points to a broken export rather than a weak model; we did not resubmit it. Two properties of the data cap strict F1: 26 surface strings carry different labels in different documents ("Union of India" as appellant, respondent or authority), which only document context can resolve, and 137 entity pairs touch with only whitespace between them, where one boundary error costs two spans. Counsel, judge and witness labels are each under 3% of entities, and each weighs one twelfth of macro-F1. **Phase 3.** A model that copies the gold previous label would score 0.749, so labels are highly sequential, yet explicit sequence models on cheap features did not help. A count-based transition prior decoded with Viterbi lowered the TF-IDF model from 0.396 to 0.31–0.35, and a document-level BiLSTM with a CRF scored 0.347. Truncation at 63 tokens cut 18–22% of sentences. Training overfitted fast: one distillation run reached 0.99 training F1 while dev loss doubled. The three later submissions scored 0.4294–0.4386 on test although their dev scores were as high as 0.4942, another case of dev selection on 10 documents failing. **Phase 4.** Dev-tuned thresholds did not transfer (run 1: 4.7 points of overstatement), and run 9 had the best dev score of the controls (0.6122) and the worst test score (0.5648). Every language-model run stayed at or below TF-IDF

on dev. **Phase 5.** Rare statutes are the weak spot: mean F1 by training-frequency quartile is 0.23, 0.33, 0.36 and 0.50. **Phase 6.** BERTScore truncates both texts at 512 RoBERTa tokens, so only about the first 400 words of a summary count, and test references average 932 words. Text beyond that point is not scored, which explains why shorter outputs and more summary context helped. A no-repeat size of 6 let the model loop by paraphrase (13% repeated word 4-grams). **Phase 7.** Of 973 dev gold cases, 449 were retrieved and 417 cannot be, because their queries have more than seven. Recall falls from 0.68 to 0.23 as queries grow longer, but hits over available slots rise from 0.65 to 0.90, so long queries are not harder. The main remaining error is citation without shared text: missed gold cases have a mean overlap z-score of 0.56 against 3.37 for hits. The neural term swapped only 58 of 826 top-7 entries (21 gold gained, 17 lost); its excerpts are chosen by the same n-gram overlap, which may limit it to evidence the lexical stage already has.

7.2 What the metrics do not measure

ROUGE and BERTScore reward overlap with one reference, not legal correctness: a summary can score well while dropping the holding, and BERTScore ignores everything after about 400 words. Strict span F1 gives no credit for a correct entity with a boundary off by one character. Statute macro-F1 weighs rare and common statutes equally, which is noisy for statutes with few test cases. Phase 7 gold labels record citation, not relevance, so an uncited but relevant precedent counts as an error.

7.3 Limitations and what you would do next

Most comparisons use one seed and test sets are small (one Phase 7 hit is worth about 0.0006 test F1), so many differences are not resolvable. Our models read at most 2,048 tokens of judgments that often exceed 10,000, the contamination audit never covered Phases 2 and 3, and no lawyer evaluated any output. Next we would score several chunks of each Phase 7 candidate so the reranker sees text the lexical stage did not select, weight the Phase 6 loss towards the span BERTScore reads, and use the hybrid's constant-size state to read whole judgments in Phases 4 and 5.

7.4 Surprising findings

Three results contradicted our expectations: the hybrid's lower pretraining loss never produced a measurable downstream gain; in Phase 7 the best standalone reranker was the worst blend component and a frozen backbone blended as well as a fine-tuned one; and a position-only signal (where citations sit in the query) was worth more test hits than any neural model (about 17 against 8).

8 Conclusion

Two choices moved the metrics: retraining the foundation model on 30 times more legal text, and combining neural models with strong lexical systems (TF-IDF in Phase 5, n-gram overlap and a learned ranker in Phase 7); in Phase 6, decoding aimed at the metric's actual window also mattered. The choice of architecture and of adaptation method did not measurably matter, so parameter-efficient methods, which halve memory, are the practical default. The main lesson for a compact legal model under a parameter and data lock is that token budget and honest evaluation matter more than architecture, and that on long legal documents exact lexical evidence is still the strongest single signal.

LLM Usage Declaration

Q1. Which Large Language Model(s), if any, did you use while completing the competition or writing this report?

Answer: Claude (Anthropic), mainly Claude Opus through the Claude Code agent, from Phase 3 onwards and for this report; ChatGPT (OpenAI) in the earlier phases.

Q2. For each row, mark Yes or No and state the purpose in one short phrase.

Part	Used LLM?	Purpose of using the LLM
Understanding concepts / papers	Yes	Explaining LoRMA, Mamba-2, BERTScore and bootstrap tests
Ideation / brainstorming at a high level	Yes	Proposing experiments and controls
Debugging small ancillary snippets	Yes	Fixing notebook and evaluation bugs
Plotting / visualization code	Yes	Figure scripts
Grammar / writing style of the report	Yes	Editing prose
Report structure / section wording	Yes	Drafting sections from our results
Designing the model architecture	Yes	Implementing the Hymba-style hybrid and encoder
Hyperparameter search / tuning	Yes	Proposing and running sweeps
Analysis of results / conclusions	Yes	Drafting analyses, checked by the author
Writing the core training / inference code	Yes	Training, inference and submission code
Other (specify)	Yes	Running Colab jobs and Codabench uploads under supervision

Q3. Did you refer to sources other than lecture slides and an LLM?

Answer: Yes: the papers in the bibliography, the course scaffold code and starter notebooks, the official evaluation scripts, and the documentation of PyTorch, scikit-learn and the `bert-score` package.

Q4. Apart from any permitted LLM use, what was *your* contribution?

My most important job in this project was to learn. I used the LLMs as implementer and tutor: they wrote most of the training, inference and analysis code and ran the experiments, while I followed each experiment, chose what to try next and when to stop, and asked for every result to be explained until I could follow the reasoning myself. I kept the written walkthroughs of each phase (the `FINDINGS` and `ANALYSIS_EXPLAINED` files in the repository) and used them to study LoRA and LoRMA, state-space layers, distillation, BERTScore truncation, threshold shrinkage and paired bootstrap tests. I also set the limits of what counted as a real result: when we found that citation traces survived the organisers' masking in Phase 7 and raised dev F1 from 0.4603 to 0.4747, I rejected using them because they improve no model. I managed the compute across Colab accounts and checked results on Codabench myself, which caught errors such as submissions and ranks the agent had missed. Two failures taught me the most. In Phase 4, thresholds tuned on dev overstated test by 2.5 points, which showed me how easily a small dev set is overfitted. In Phase

7, the reranker trained with random negatives was the best model on its own but the worst in the blend, which taught me that a component's value depends on what the rest of the system already knows. A late attempt to score whole judgments in windows with the existing rerankers also gave no dev gain, and I chose to stop rather than spend the last hours on it. Overall I learned more from these controlled comparisons and negative results than from the leaderboard scores.

References

[1] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger, and Y. Artzi, "BERTScore: Evaluating text generation with BERT," *International Conference on Learning Representations*, 2020.

[2] P. Kalamkar, A. Tiwari, A. Agarwal, S. Karn, S. Gupta, V. Raghavan, and A. Modi, "Corpus for automatic structuring of legal documents," in *Proceedings of LREC*, 2022.

[3] V. Malik, R. Sanjay, S. K. Nigam, K. Ghosh, S. K. Guha, A. Bhattacharya, and A. Modi, "ILDC for CJPE: Indian legal documents corpus for court judgment prediction and explanation," in *Proceedings of ACL*, 2021.

[4] N. Shazeer, "GLU variants improve transformer," *arXiv preprint arXiv:2002.05202*, 2020.

[5] B. Zhang and R. Sennrich, "Root mean square layer normalization," in *Advances in Neural Information Processing Systems*, 2019.

[6] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu, "RoFormer: Enhanced transformer with rotary position embedding," *Neurocomputing*, vol. 568, 2024.

[7] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "GQA: Training generalized multi-query transformer models from multi-head checkpoints," in *Proceedings of EMNLP*, 2023.

[8] DeepSeek-AI, "DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model," *arXiv preprint arXiv:2405.04434*, 2024.

[9] X. Dong, Y. Fu, S. Diao, W. Byeon, Z. Chen, A. S. Mahabaleshwarkar, S.-Y. Liu, M. Van Keirsbilck, M.-H. Chen, Y. Suhara, *et al.*, "Hymba: A hybrid-head architecture for small language models," *arXiv preprint arXiv:2411.13676*, 2024.

[10] T. Dao and A. Gu, "Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality," in *International Conference on Machine Learning*, 2024.

[11] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of NAACL*, 2019.

[12] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *International Conference on Learning Representations*, 2019.

[13] J. Hoffmann, S. Borgeaud, A. Mensch, *et al.*, "Training compute-optimal large language models," *arXiv preprint arXiv:2203.15556*, 2022.

[14] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *International Conference on Learning Representations*, 2022.

[15] H. Bihany, S. Patel, and A. Modi, "LoRMA: Low-rank multiplicative adaptation for LLMs," *arXiv preprint arXiv:2506.07621*, 2025.

[16] H. Liu, D. Tam, M. Muqeeth, J. Mohta, T. Huang, M. Bansal, and C. Raffel, "Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning," in *Advances in Neural Information Processing Systems*, 2022.

[17] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015.

[18] J. Lafferty, A. McCallum, and F. Pereira, "Conditional random fields: Probabilistic models for segmenting and labeling sequence data," in *Proceedings of ICML*, 2001.

[19] R. Nogueira and K. Cho, "Passage re-ranking with BERT," *arXiv preprint arXiv:1901.04085*, 2019.

[20] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, 2009.

[21] F. Pedregosa, G. Varoquaux, A. Gramfort, *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, 2011.

[22] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems*, 2017.